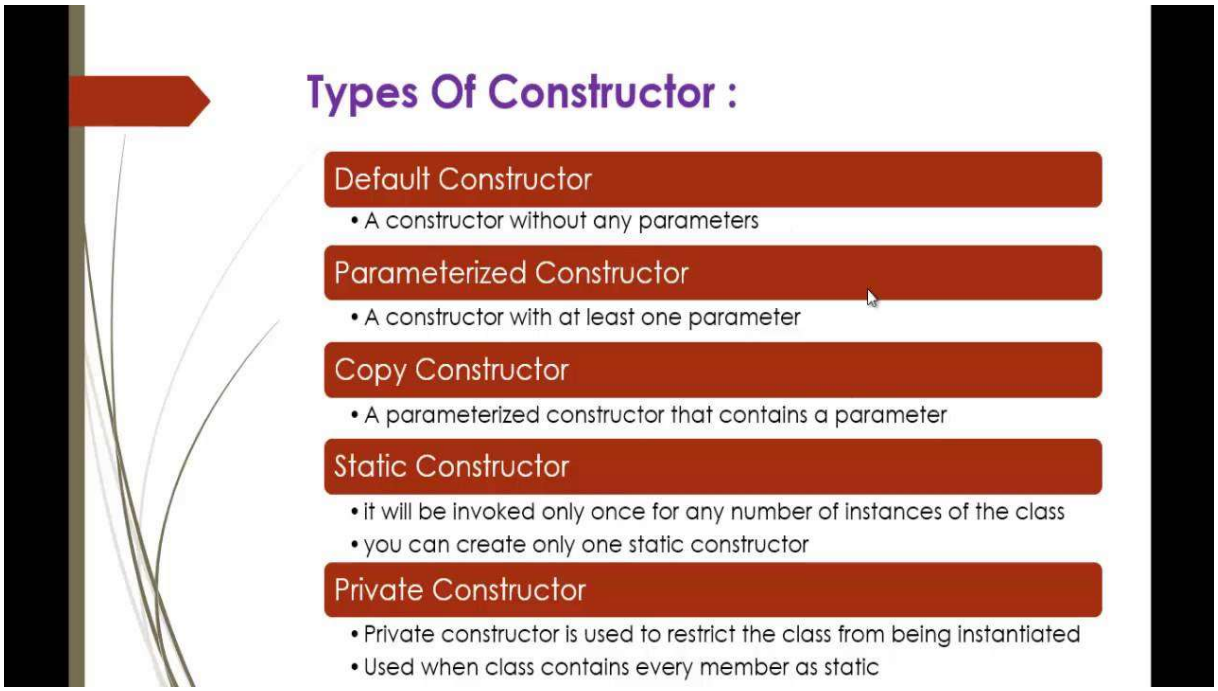
OOP'S Concepts

Constructor in Java

A constructor in Java is a special type of method that is automa cally called when an object of a class is created.

It is mainly used to ini alize objects (assign values to instance variables).

Types of Constructors in Java



2. Parameterized Constructor

- **A constructor that accepts arguments/parameters to initialize objects with specific values.**
- **Used when we want to assign custom values during object creation.**
- **Can have any number of parameters.**
- **Supports constructor overloading.**

Syntax :-

```
class ClassName {
   ClassName(type1 param1, type2 param2, ...) {
      // body of constructor
   }
}
```

Example :-

```
package oops_condtructors;


public class paramethirized_cons {
        paramethirized_cons(int a)
        {
                System.out.println("hello pune");
        }
        paramethirized_cons(int a,int b,int c)
        {
                System.out.println("mul is : "+(a*b*c));
        }
        paramethirized_cons(int a,int b,int c,int d)
```

```
        {
                System.out.println("add is : "+(a+b+c+d));
        }

        public static void main(String[] args) {
                paramethirized_cons obj=new paramethirized_cons(5);
                paramethirized_cons obj2=new paramethirized_cons(5,4,2);
                paramethirized_cons obj3=new paramethirized_cons(5,40,4,8);
        }
}
```

Output :-

hello pune

mul is : 40

add is : 57

3. Copy Constructor in Java

Java does not provide a built-in copy constructor (unlike C++).
But we can define one manually to copy values from one object into another.

Syntax:-

```
class ClassName {
   ClassName(ClassName obj) {
      // copy object values here
   }
}
```

Example :-

```
package oops_condtructors;


public class employee {
	int emp_id;
	String emp_name;
	employee(int emp_id,	String emp_name)//para cons
	{
		this.emp_id=emp_id;
		this.emp_name=emp_name;
		System.out.println("emp id :"+this.emp_id);
		System.out.println("emp name is :"+this.emp_name);


	}
	employee(employee e)//copy constructor
	{
		this.emp_id=e.emp_id;
		this.emp_name=e.emp_name;
		System.out.println("copy of emp id is :"+this.emp_id);
		System.out.println("copy of emp name is :"+this.emp_name);
	}
	public static void main(String[] args) {
		employee emp1=new employee(25,"sahil");//call para cons
		System.out.println();
		employee emp2=new employee(emp1);//call copy cons
	}
}
```

Output :-

emp id : 25

emp name is : sahil

copy of emp id is : 25

copy of emp name is : sahil

4. **Private Constructor in Java**

- A **private constructor** is declared as private.
- It **cannot be accessed from outside the class**, so objects of that class **cannot be created directly**.
- **Restricting object creation**
- **Utility classes** (like Math in Java)
- **Singleton Design Pattern** (only one object allowed)

Syntax:

```
class ClassName {
   private ClassName() {
      // initialization code
   }
}
```

Example :-

```
class Demo {
   private Demo() {   // private constructor
      System.out.println("Private constructor invoked");
   }
```

```
    public static void show() {
        System.out.println("Static method can be called without object creation");
    }
}

public class Main {
    public static void main(String[] args) {
        // Demo d = new Demo();
        Demo.show
    }
}
```

Output:

Static method can be called without object creation